# Testiranje opterećenja

Napredne veb tehnologije

Ana Šinik SV11/2022
Nataša Kašiković SV69/2022
Lazar Nagulov SV61/2022

Novi Sad, 2026

**Uvod** ..... **2**
**Pregled sistema** ..... **3**
Opis testirane aplikacije ..... 3
Testno okruženje ..... 4
**Priprema testnih podataka** ..... **5**
**Metodologija testiranja** ..... **6**
Korišćeni alat ..... 6
Profil opterećenja ..... 6
**Pregled testiranih endpoint-a** ..... **7**
**Rezultati testiranja** ..... **9**
Dobavljanje svih marki vozila ..... 11
Dobavljanje svih modela vozila po marki ..... 12
Pretraga vozila ..... 14
Dobavljanje podataka o vozilu ..... 15
Dobavljanje poslednje poznate lokacije vozila ..... 17
Dobavljanje svih država ..... 18
Dobavljanje gradova po državi ..... 20
Verifikacija registrovane firme ..... 21
Dobavljanje neobrađenih firmi ..... 23
**Zaključak** ..... **25**
**Appendix** ..... **26**
Bekend konfiguracija tokom testiranja ..... 26

# Uvod

Cilj ovog dokumenta je da prikaže rezultate testiranja performansi bekend aplikacije korišćenjem alata Locust (https://locust.io/). Testiranje je fokusirano na najčešće korišćene i performansno kritične REST endpoint-e.

Bekend aplikacija predstavlja serversku komponentu informacionog sistema za upravljanje poslovnim procesima kompanije, uključujući proizvodnju, skladištenje i distribuciju proizvoda, pri čemu se očekuje istovremen pristup većeg broja korisnika. Testiranje je sprovedeno kao test opterećenja sa postepenim povećanjem broja konkurentnih korisnika, u cilju analize ponašanja sistema i identifikacije potencijalnih uskih grla. Posebna pažnja posvećena je vremenu odziva i propusnosti sistema.

Projekat je javan i dostupan na sledećem GitLab repozitorijumu: https://gitlab.com/lazarnagulov/euro-supply.

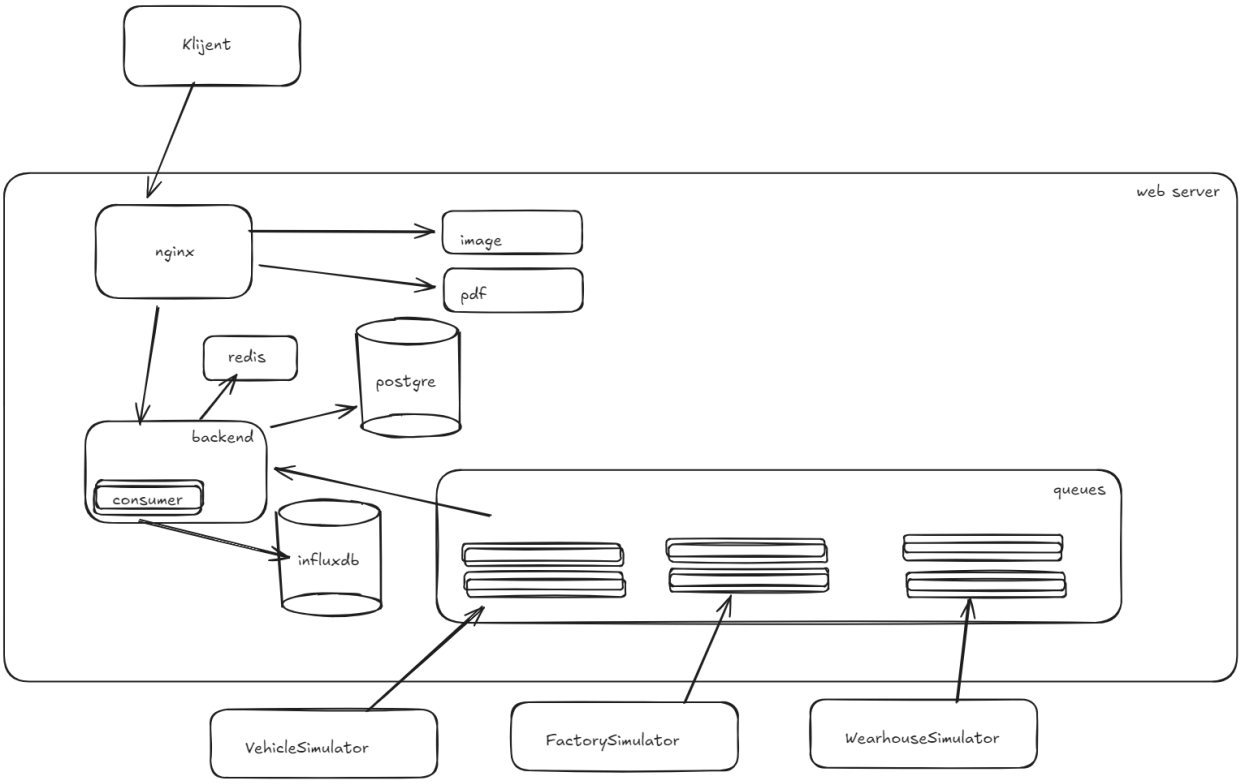
# Pregled sistema

## Opis testirane aplikacije

EuroSupply je veb aplikacija čiji je bekend deo razvijen korišćenjem Spring Boot radnog okvira (engl. *framework*)(https://spring.io/), dok je korinički interfejs implementiran pomoću React biblioteke (https://react.dev/). Simulatori fabrika, dostavnih vozila i magacina razvijeni su u programskom jeziku Go (https://go.dev/).

Podaci sistema se skladište u dve odvojene baze podataka: relacionoj bazi PostgreSQL (https://www.postgresql.org/) i bazi podataka vremenskih serija InfluxDB (https://www.influxdata.com/). Za asinhronu komunikaciju između komponenti sistema koristi se RabbitMQ (https://www.rabbitmq.com/). Redis (https://redis.io/) se koristi za keširanje podataka, dok je Nginx (https://nginx.org/) zadužen za serviranje statičkog sadržaja i prosleđivanje zahteva ka backend aplikaciji.

Skica sistema se nalazi na Slici 1.




Slika 1 - skica sistema

# Testno okruženje

Testovi su izvršeni na 3 različite lokalna mašine. Svaka komponenta sistema je pokrenuta lokalno preko dokera (https://www.docker.com/) . Na svakom testiranom endpointu je navedeno na kojoj mašini je testiran.

**Mašina 1 (Student 3)**:

Operativni sistem: Windows 10
Processor: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz
RAM: 16.0 GB

**Napomena: Student 1 i Student 2 će testirati svoje endpoint-ove i dopuniti specifikacije njihovih mašina pre finalne verzije izveštaja.**

# Priprema testnih podataka

Relaciona baza podataka sadrži 2 miliona kupaca (od kojih jedan ima najmanje 100 firmi), 2 miliona registrovanih i prihvaćenih firmi, 1 milion menadžera, 20.000 fabrika, 40.000 magacina i najmanje 100.000 dostavnih vozila. Podskup kompanija je označen kao *pending* kako bi se omogućilo ponovljivo testiranje verifikacije registrovane firme. Države, gradovi, marke vozila, modeli vozila i kategorije proizvoda predstavljaju šifrarnike sa unapred definisanim skupovima podataka. Sistem sadrži 47 država i 141 grad (po 3 grada po državi), 20 marki vozila sa ukupno 60 modela (po 3 modela po marki), kao i 25 kategorija proizvoda.

Baza podataka vremenskih serija sadrži istorijske podatke o proizvodnji, skladištenju i transportu. Za najmanje 1.000 fabrika obuhvaćeni su realni podaci o proizvodnji za prethodne četiri godine. Svaka fabrika proizvodi najmanje deset različitih proizvoda, a podaci o proizvodnji se evidentiraju dva puta dnevno.

Baza takođe sadrži podatke o merenju temperature za najmanje 100 magacina u periodu od prethodne dve godine. Svaki magacin je podeljen na najmanje tri sektora, a merenja temperature su beležena u pravilnim vremenskim intervalima tokom dana.

Za najmanje 100 dostavnih vozila sačuvani su podaci o pređenoj distanci između dva uzastopna javljanja za prethodnih pet godina. Podaci o kretanju vozila prikupljani su više puta dnevno, što omogućava detaljno praćenje transporta.

Program za upis podataka u bazu vremenskih serija implementiran je u okviru simulatora. Detaljno uputstvo za korišćenje skripte dostupno je u README datoteci. U *development* režimu, podaci u relacionoj bazi se automatski popunjavaju.

# Metodologija testiranja

## Korišćeni alat

U svrhu testiranja opterećenosti sistema korišćen je alat Locust. Locust se koristi za simuliranje velikog broja korisnika koji pristupaju veb aplikaciji istovremeno. Osnovna ideja je stvaranje virtualnih korisnika, poznatih kao "korisnički agenti", koji šalju zahteve veb aplikaciji kako bi se procenila njena izvedba pod određenim opterećenjem.

Testni scenariji su napisani u programskom jeziku Python (https://www.python.org/).

## Profil opterećenja

Testovi opterećenja su sprovedeni korišćenjem postepenog (engl. *step-based*) profila, pri čemu je broj istovremenih korisnika postepeno povećavan kako bi se ispitalo ponašanje sistema pri rastućem opterećenju.

Tokom jednog testiranja aktivan je samo jedan endpoint[1]. Na taj način obezbeđeno je da rezultati jasno prikažu performanse konkretnog endpointa, bez uticaja ostalih API poziva.

Osnovni nivoi opterećenja tokom testiranja bili su 10, 25, 50, 100 i 200 istovremenih korisnika. U zavisnosti od prirode i složenosti endpointa, kao i od resursa koje koristi (npr. keširani podaci), ovi nivoi su po potrebi prilagođavani i proširivani kako bi se ispitali realni kapaciteti sistema. Na primer, endpointi koji se oslanjaju isključivo na keš testirani su i pri značajno većem broju korisnika.

Svaki nivo opterećenja trajao je između 30 i 90 sekundi, što je omogućilo da se performanse sistema stabilizuju i da se dobiju relevantni podaci o vremenu odziva i propusnosti. Nakon svakog testa, statistike u Locustu su resetovane kako se podaci iz različitih faza ne bi međusobno mešali.

[1] Ukoliko je endpoint zahtevao autentifikaciju i autorizaciju, dodatno je slat zahtev za prijavu radi pribavljanja autentifikacionog tokena.

# Pregled testiranih endpoint-a

Tabela 1 sadrži listu svih endpoint-ova planiranih za testiranje. Endpoint-ovi označeni sa (*) su trenutno testirani i dokumentovani u ovom izveštaju. Preostali endpoint-ovi biće testirani od strane Student 1 i Student 2.

Svakom endpointu je izostavljen prefiks /api/v1/.

| Endpoint | Metoda | Uloga | Opis |
|---|---|---|---|
| /vehicles/{id}/distances(*) | GET | Menadžer[2] | Dobavljanje grafika ukupne pređene distance |
| /vehicles/brands(*) | GET | Ulogovani korisnik[3] | Dobavljanje svih marki vozila |
| /vehicles/brands/{id}/model(*) | GET | Ulogovani kornisk | Dobavljanje svih modela vozila po marki |
| /vehicles/search(*) | GET | Menadžer | Pretraga vozila |
| /vehicles/{id}(*) | GET | Menadžer | Dobavljanje podataka o vozilu |
| /vehicles/{id}/location(*) | GET | Menadžer | Dobavljanje poslednje poznate lokacije vozila |
| /countries(*) | GET | Ulogovani korisnik | Dobavljanje svih država (u Evropi) |
| /countries/{id}/cities(*) | GET | Ulogovani korisnik | Dobavljanje svih gradova po državi |
| /companies/{id}(*) | PATCH | Menadžer | Verifikacija registrovane firme |
| /companies/pending(*) | GET | Menadžer | Dobavljanje neobrađenih firmi |
| /categories | GET | Ulogovani korisnik | Dobavljanje kategorije proizvoda |
| /products | POST | Menadžer | Kreiranje proizvoda |
| /products/search | GET | Ulogovani korisnik | Pretraga proizvoda |

[2] Administrator ima iste mogućnosti kao i menadžer
[3] Ulogovani korisnik može da bude kupac, menadžer i administrator

| Endpoint | Metoda | Uloga | Opis |
|---|---|---|---|
| /products/on-sale/search | GET | Kupac | Pretraga svih proizvoda koji su na prodaju |
| /products/order | GET | Kupac | Poručivajne proizvoda |
| /users/managers | GET | Administrator | Pretraga menadžera |
| /factories/search | GET | Menadžer | Pretraga fabrika |
| /factories/{id}/products | GET | Menadžer | Dobavljanje svih proizvoda koje fabrika proizvodi |
| /factories | POST | Menadžer | Kreiranje fabrike |
| /factories | GET | Menadžer | Dobavljanje fabrika |
| /factories/{id}/status | GET | Menadžer | Dobavljanje trenutnog statusa fabrike |
| /auth/login | POST | Neulogovani korisnik | Prijava korisnika na sistem |
| /wearhouse/search | GET | Menadžer | Pretraga magacina |
| /wearhouse | POST | Menadžer | Kreiranje magacina |
| /wearhouse/{id}/status | GET | Menadžer | Dobavljanje trenutnog statusa magacina |
| /users/registration | POST | Neulogovani korisnik | Registracija korisnika na sistem |
| /users/verification | PATCH | Neulogovani korisnik | Verifikacija korisničkog naloga |

Tabela 1 - spisak svih endpointova koji se testiraju

# Rezultati testiranja

Rezultati testiranja izraženi su kroz prosečno vreme odziva (engl. *Average Response Time*, Avg RT) i 95. percentil vremena odziva (P95 RT)[4], pri čemu su obe metrike izražene u milisekundama.

## Dobavljanje grafika ukupne pređene distance

### Endpoint: /api/v1/vehicles/{id}/distances
### Metoda: GET
### Testirano na mašini 1

Endpoint za dobavljanje grafika ukupne pređene distance poziva se prilikom učitavanja stranice vozila, kao i pri svakoj promeni vremenskog perioda koji korisnik želi da prikaže na grafiku. Zbog obrade i agregacije podataka vremenskih serija, ovaj endpoint ima značajan uticaj na performanse sistema.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 29,63 | 29,65 |
| 50 | 5 | 23,21 | 23,20 |
| 200 | 5 | 32,31 | 32,32 |
| 750 | 20 | 99,96 | 99,97 |
| 1250 | 50 | 1115,17 | 1114,66 |

Tabela 2 - rezultati dobavljanja grafika ukupne pređene distance

Rezultati testiranja endpoint-a za dobavljanje pređenih distanci vozila pokazuju stabilne performanse pri malom i srednjem opterećenju. Za do 200 istovremenih korisnika prosečno vreme odziva i 95. percentil ostaju relativno niski i gotovo identični, što ukazuje na ujednačeno vreme izvršavanja Flux upita i odsustvo značajnih varijacija u obradi zahteva.

Pri povećanju broja korisnika na 750 dolazi do primetnog rasta vremena odziva, što ukazuje na povećano opterećenje InfluxDB baze i pojavu čekanja zahteva u redovima za obradu. U ovom režimu rada sistem i dalje funkcioniše stabilno, ali se približava granicama svojih performansi.

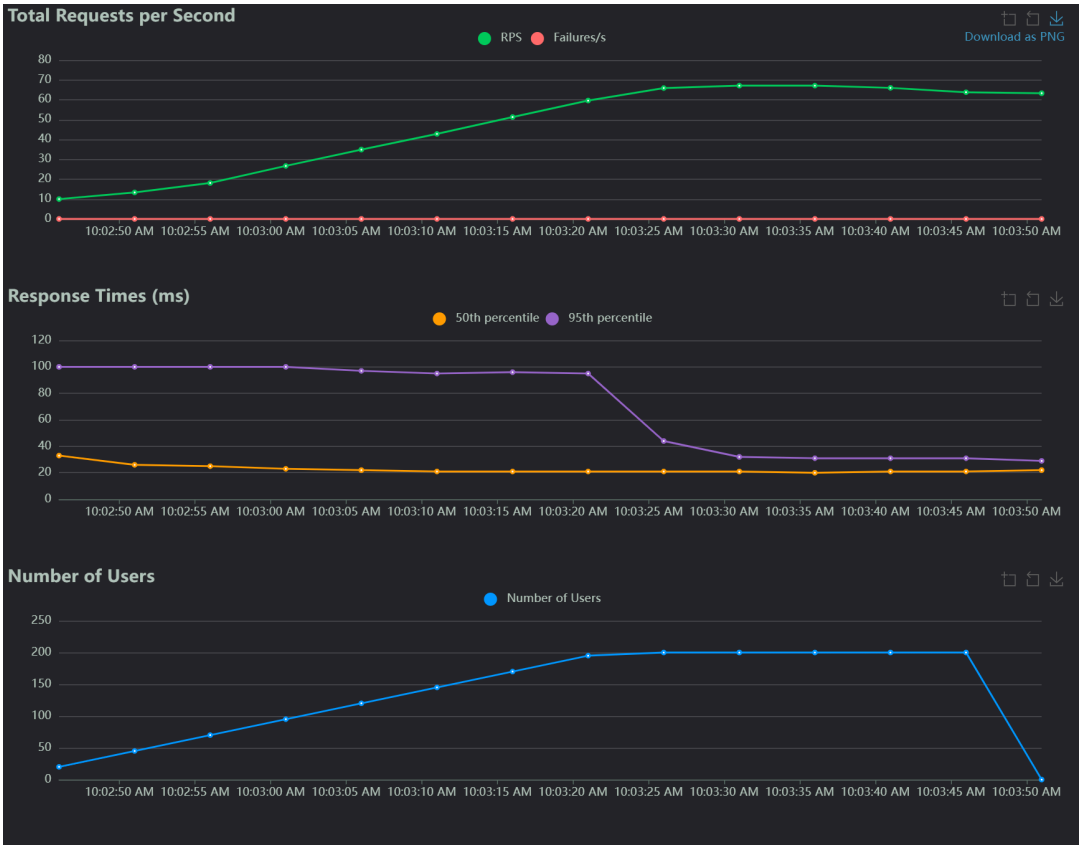
[4] 95. percentil predstavlja vreme odziva ispod kog se nalazi 95% svih merenja.

Pri ekstremnom opterećenju od 1250 istovremenih korisnika dolazi do naglog pogoršanja performansi, gde prosečno vreme odziva i 95. percentil prelaze jednu sekundu. Ovo ponašanje ukazuje na zasićenje resursa usled velikog broja paralelnih agregacionih upita nad vremenskim serijama podataka, što predstavlja očekivano ograničenje ovog tipa operacije.

Tokom testiranja uočen je nedostatak u implementaciji validacije postojanja vozila, gde se metoda za proveru postojanja vozila direktno oslanjala na pristup bazi podataka, bez korišćenja keširanih podataka. Nakon uvođenja keširanja ponovljena su testiranja endpoint-a pod identičnim uslovima.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 50 | 5 | 25,7 | 25,68 |
| 200 | 5 | 21,59 | 21,59 |
| 750 | 20 | 37,93 | 37,93 |
| 1250 | 50 | 173,07 | 173,007 |

Tabela 3 - rezultati dobavljanja grafika ukupne pređene distance sa keširanjem




Grafik 1 -Dobavljanje grafika ukupne pređene distance sa keširanjem (200 korisnika)

Rezultati testiranja pokazuju stabilnije ponašanje sistema pod opterećenjem u poređenju sa prethodnom implementacijom. Pri malom broju korisnika (10), prosečno vreme odziva je relativno veće jer je keš još uvek hladan (većina zahteva mora direktno pristupiti bazi podataka, a prvi zahtevi dodatno upisuju podatke u keš). Sa rastom broja korisnika, keš se postepeno popunjava i sve veći deo zahteva se služi direktno iz njega, što se reflektuje u smanjenju prosečnog vremena odziva već od 50 korisnika nadalje.

U poređenju sa prethodnom implementacijom, izmenjeni pristup pokazuje značajno bolje ponašanje pri većem opterećenju, što potvrđuje pozitivan uticaj izmene na skalabilnost sistema.

# Dobavljanje svih marki vozila

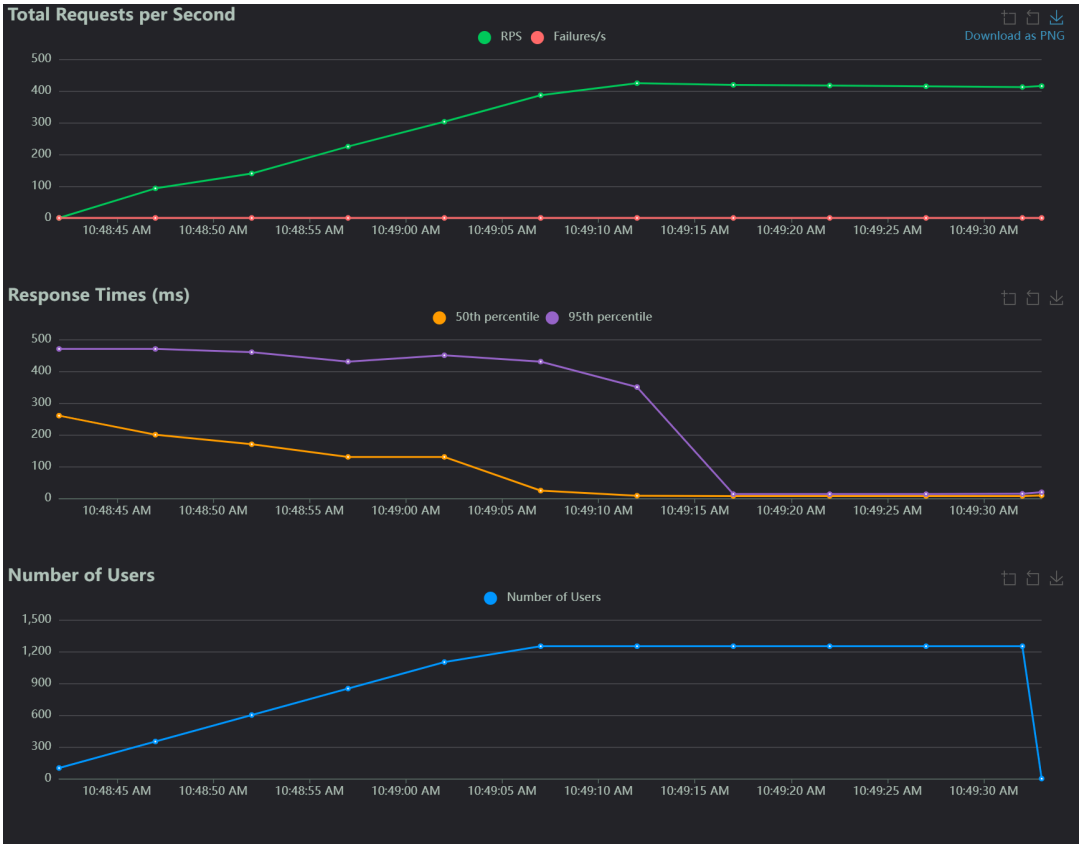
## Endpoint: /api/v1/vehicles/brands
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje marki vozila se automatski poziva pri pristupu formi za kreiranju, ažuriranju i pretrazi vozila.

S obzirom na to da se podaci o marki veoma retko menjaju, prilikom prvog poziva svi podaci se upisuju u Redis keš sa vremenom važenja (engl. *TTL*) od 12 sati. Na taj način se smanjuje opterećenje baze podataka i obezbeđuje brz odgovor sistema za veliki broj istovremenih korisnika.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 25 | 2 | 9,88 | 13 |
| 50 | 5 | 9,92 | 13 |
| 400 | 10 | 10 | 14 |
| 750 | 20 | 16,64 | 94 |
| 1250 | 50 | 50,95 | 330 |

Tabela 4 - rezultati dobavljanja svih marki vozila




Grafik 2 - grafik dobavljanja svih marki vozila (1250 korisnika)

Na osnovu prikazanih rezultata može se uočiti da endpoint za dobavljanje modela automobila pokazuje vrlo dobre performanse pri malom i srednjem opterećenju. Za 50 i 400 istovremenih korisnika prosečno vreme odziva ostaje ispod 10 ms. Ovakvo ponašanje je očekivano s obzirom na to da se podaci keširaju u Redis-u i da se pri većini zahteva ne ostvaruje direktan pristup bazi podataka.

Povećanjem broja korisnika na 750 dolazi do umerenog rasta vremena odziva, pri čemu prosečno vreme odziva ostaje relativno nisko, dok vrednosti 95. percentila rastu brže. Ovo ukazuje na pojavu zagušenja resursa i povećano čekanje pojedinih zahteva u redovima za obradu, iako sistem i dalje uspešno obrađuje zahteve bez značajnog pada propusnosti.

# Dobavljanje svih modela vozila po marki

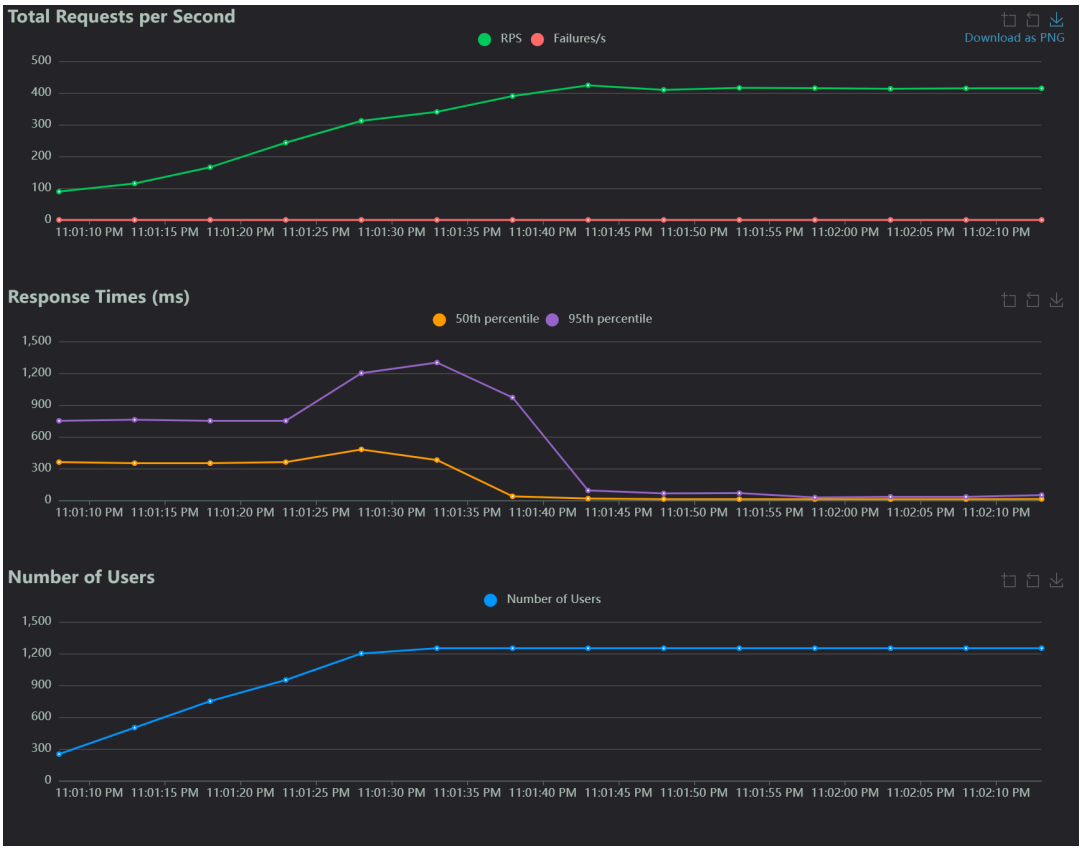
## Endpoint: /api/v1/vehicles/brands/{id}/models
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje modela vozila po marki se automatski poziva pri selekciji marke u formi za registraciju firme, kreiranju, ažuriranju i pretrazi vozila.

S obzirom na to da se podaci o modelima veoma retko menjaju, prilikom prvog poziva svi podaci se upisuju u Redis keš sa vremenom važenja (engl. *TTL*) od 12 sati. Na taj način se smanjuje opterećenje baze podataka i obezbeđuje brz odgovor sistema za veliki broj istovremenih korisnika.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 23,29 | 43,4 |
| 25 | 2 | 16,11 | 24,65 |
| 50 | 5 | 15,29 | 19.6 |
| 400 | 10 | 35,72 | 209,5 |
| 1250 | 50 | 125,7 | 678 |

Tabela 5 - rezultati dobavljanja svih modela vozila po marki




Grafik 3 - grafik dobavljanja modela vozila po marki (1250 korisnika)

Rezultati testiranja endpoint-a za dobavljanje modela vozila po marki pokazuju zanimljivo ponašanje u uslovima rastućeg opterećenja. Pri malom broju korisnika prosečno vreme odziva je relativno veće, što se može pripisati inicijalnom punjenju keša i manjem broju ponovljenih zahteva za istim podacima.

Povećanjem broja istovremenih korisnika dolazi do smanjenja prosečnog vremena odziva, što ukazuje na efekat zagrevanja keša i povećan broj pogodaka u kešu.

Pri većem opterećenju, od 1250 korisnika, uočava se postepeni rast vrednosti 95. percentila vremena odziva, što ukazuje na pojavu zadržavanja pojedinih zahteva usled ograničenih serverskih resursa.

# Pretraga vozila

## Endpoint: /api/v1/vehicles/search
## Metoda: GET
## Testirano na mašini 1

Endpoint za pretragu vozila koristi se na stranici za pregled i upravljanje vozila. Poziva se svaki put kada menadžer ili administrator unese ili izmeni parametre u filter sekciji, kao i prilikom promene stranice rezultata uz aktivne filtere.

Pretraga omogućava filtriranje vozila po više kriterijuma, uključujući registracioni broj, marku, model i ograničenje težine. Zbog čestih interakcija korisnika sa filterima, ovaj endpoint može generisati veliki broj zahteva u kratkom vremenskom periodu i predstavlja jedan od ključnih endpointa sa aspekta performansi i skalabilnosti sistema.

Radi obezbeđivanja efikasne i skalabilne pretrage nad velikim brojem vozila, primenjene su odgovarajuće tehnike optimizacije na nivou baze podataka. Za pretragu po registracionom broju korišćen je GIN trigram indeks, koji omogućava brzu i efikasnu delimičnu pretragu teksta bez potrebe za sekvencijalnim skeniranjem cele tabele. Pored toga, definisani su kompozitni i pojedinačni indeksi nad atributima koji se najčešće koriste u filterima (marka, model i maksimalna težina).

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 100 | 5 | 8,11 | 8,61 |
| 750 | 20 | 13,42 | 20,96 |
| 1250 | 50 | 33,04 | 65,49 |
| 3000 | 200 | 2096,14 | 3331,48 |

Tabela 6 - rezultati pretrage vozila sa indeksom

U cilju evaluacije uticaja indeksiranja na performanse pretrage, sprovedeno je dodatno testiranje nakon uklanjanja svih indeksa nad tabelom vozila. Rezultati pokazuju da pri manjem broju korisnika razlike u vremenu odziva nisu značajne, dok sa porastom broja istovremenih zahteva dolazi do izraženog povećanja vremena odziva u slučaju neindeksiranih upita. Posebno je primetno povećanje 95. percentila pri velikom opterećenju, što ukazuje na sekvencijalno skeniranje tabele i nemogućnost sistema da efikasno obradi veliki broj paralelnih zahteva bez odgovarajućih indeksa.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 750 | 20 | 27,14 | 36,99 |
| 1250 | 50 | 140,41 | 243,97 |
| 3000 | 200 | 5074,18 | 6600,75 |

Tabela 7 - rezultati pretrage vozila bez indeksom

Grafik 4 - poređenje performansi sa indeksom i bez indeksa (1250 korisnika)

# Dobavljanje podataka o vozilu

## Endpoint: /api/v1/vehicles/{id}
## Metoda: GET
## Testirano na mašini 1

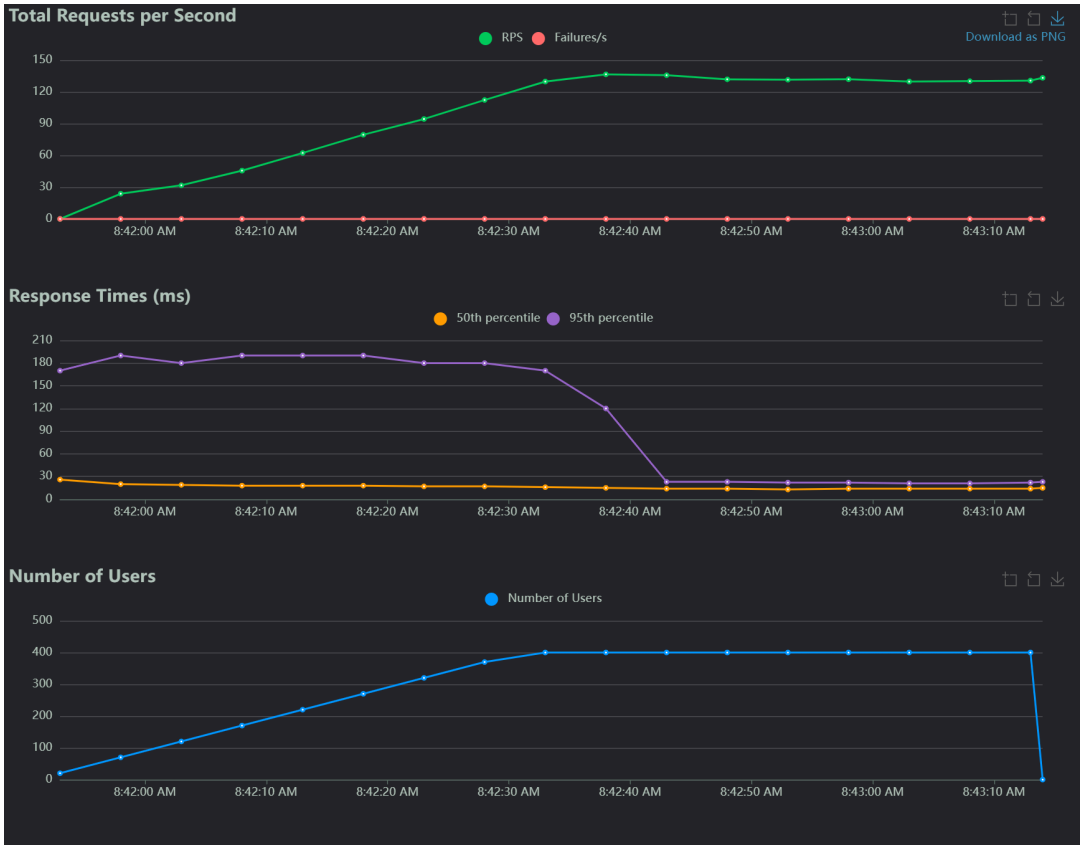
Endpoint za dobavljanje podataka o vozilu poziva se prilikom učitavanja stranice sa detaljima vozila. Njegova svrha je da obezbedi osnovne informacije o izabranom vozilu, uključujući statične podatke i trenutno stanje relevantno za prikaz korisniku.

Dobavljanje grafika ukupne pređene distance i podataka o dostupnosti vozila su odvojeni endpointi.

S obzirom na to da je stranica sa detaljima vozila jedna od najčešće korišćenih u sistemu, ovaj endpoint se često poziva i može biti istovremeno korišćen od strane velikog broja korisnika. Zbog toga je posebno značajan za procenu vremena odziva sistema i efikasnosti mehanizama za optimizaciju pristupa podacima (npr. keširanje).

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 31,98 | 31,96 |
| 25 | 2 | 19,23 | 19,26 |
| 200 | 10 | 17,7 | 17,84 |
| 750 | 20 | 36,32 | 38,12 |
| 2000 | 100 | 1710,9 | 1889,96 |

Tabela 8- rezultati dobavljanja podataka o vozilu




Grafik 5 - grafik dobavljanja podataka o vozilu (400 korisnika)

Rezultati testiranja pokazuju da keširanje odgovora značajno smanjuje prosečno vreme odziva endpointa za dobavljanje podataka o vozilu. Čak i pri velikom broju istovremenih korisnika, sistem zadržava prihvatljivo vreme odziva, dok se povećanje 95. percentila javlja tek pri ekstremnim opterećenjima, što je očekivano ponašanje usled ograničenja sistemskih resursa.

# Dobavljanje poslednje poznate lokacije vozila

## Endpoint: /api/v1/vehicles/{id}/location
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje poslednje poznate lokacije vozila se koristi na stranici sa detaljima vozila, gde se podaci periodično osvežavaju (engl. *polling*) kako bi korisniku bila prikazana ažurna lokacija. Zbog učestalih poziva i potencijalno velikog broja istovremenih korisnika, ovaj endpoint predstavlja značajan deo opterećenja sistema.

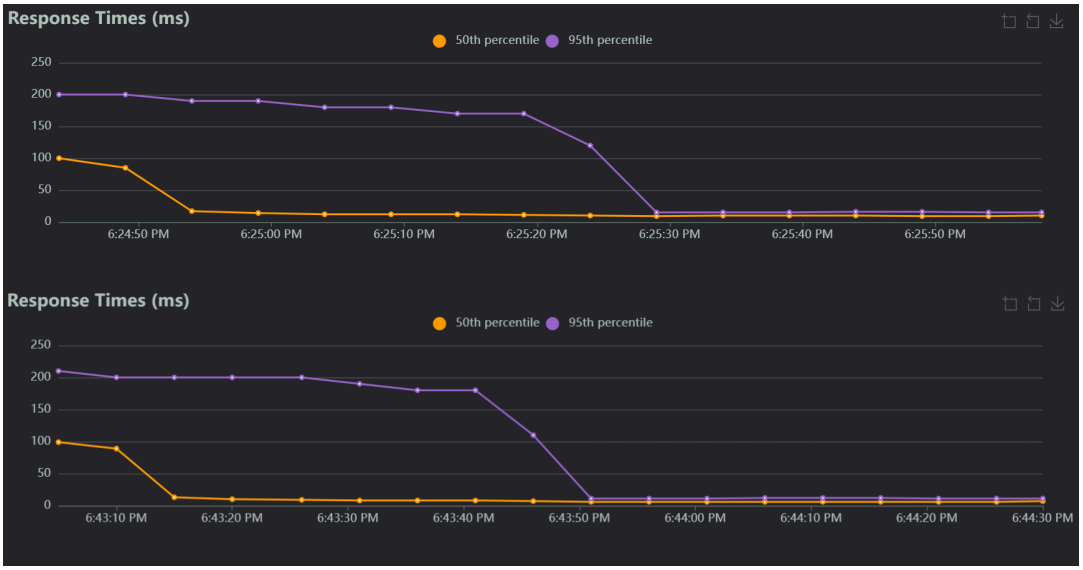
S obzirom na to da se podaci o lokaciji vozila često menjaju, poslednja poznata lokacija se čuva u posebnoj tabeli koja je u relaciji 1 na 1 sa entitetom vozila. Ovi podaci se upisuju u keš sa vremenskim ograničenjem važenja (engl. TTL) od 30 sekundi, čime se smanjuje broj direktnih pristupa bazi podataka i omogućava brži odgovor sistema pri velikom broju istovremenih zahteva. Testiranje je sprovedeno sa uključenim i isključenim keširanjem.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 25 | 2 | 13,83 | 13,82 |
| 200 | 10 | 11,27 | 11,32 |
| 750 | 20 | 17,14 | 17,76 |
| 1250 | 50 | 45,65 | 51,88 |

Tabela 9 - rezultati dobavljanja poslednje poznate lokacije vozila sa kešom

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 750 | 20 | 13,66 | 14,4 |
| 1250 | 50 | 45,26 | 51,94 |

Tabela 10 - rezultati dobavljanja poslednje poznate lokacije vozila bez keša




Grafik 6 - poređenje performansi sa kešom i bez keša (750 korisnika)

Upoređivanjem rezultata testiranja sa uključenim i isključenim keširanjem uočen je izostanak poboljšanja performansi endpointa. Pri srednjem opterećenju (750 korisnika) keširanje je čak dovelo do blagog povećanja vremena odziva, dok su pri većem opterećenju (1250 korisnika) rezultati praktično identični. S obzirom na veliki broj vozila u sistemu i ravnomernu raspodelu zahteva po ID-jevima, keširanje u ovom scenariju ne ostvaruje dovoljan stepen ponovne upotrebe da bi donelo merljivu korist.

# Dobavljanje svih država

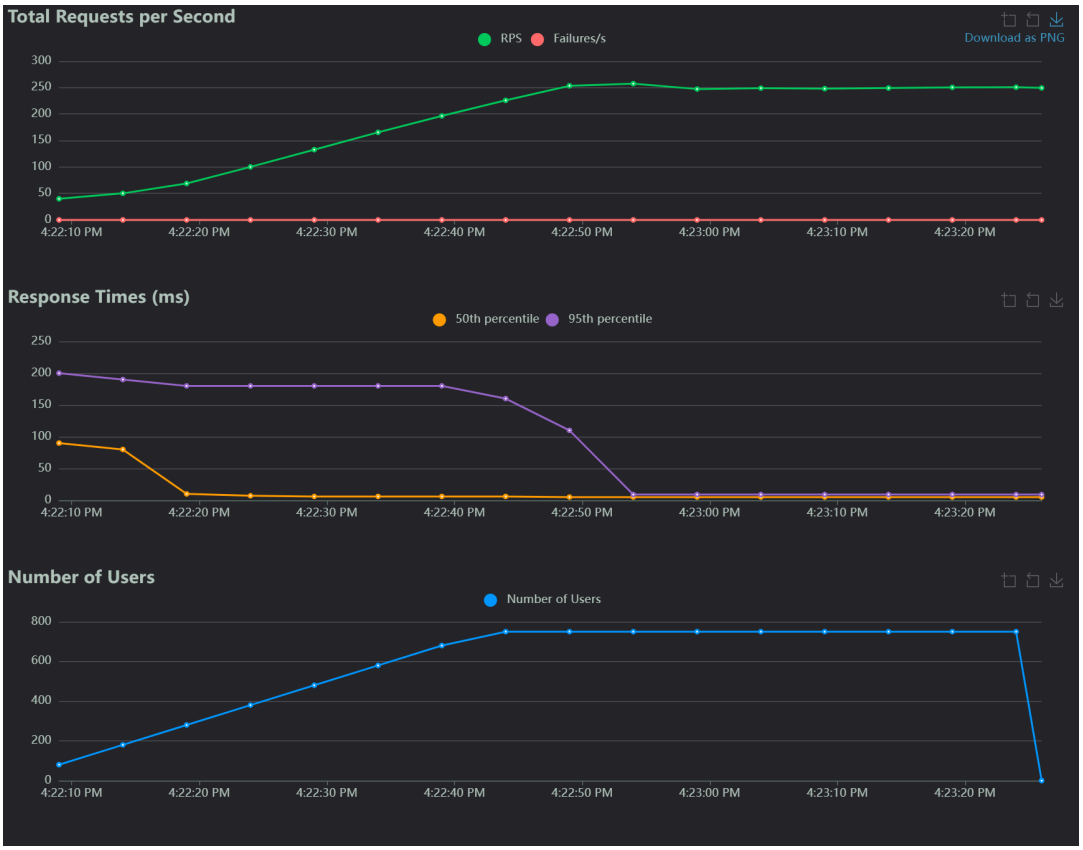
## Endpoint: /api/v1/countries
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje država se automatski poziva pri pristupu formi za registraciju firme, kreiranju, ažuriranju i pretrazi fabrika, kreiranju, ažuriranju i pretrazi magacina.

S obzirom na to da se podaci o državama veoma retko menjaju, prilikom prvog poziva svi podaci se upisuju u keš sa vremenom važenja (engl. *TTL*) od 12 sati. Na taj način se smanjuje opterećenje baze podataka i obezbeđuje brz odgovor sistema za veliki broj istovremenih korisnika.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 50 | 5 | 6,58 | 11 |
| 400 | 10 | 7,09 | 11 |
| 750 | 20 | 12,23 | 83 |
| 2000 | 100 | 882,36 | 7300 |

Tabela 11 - rezultati dobavljanja svih država




Grafik 7 - grafik dobavljanja svih država (750 korisnika)

Na osnovu prikazanih rezultata može se uočiti da endpoint za dobavljanje država pokazuje vrlo dobre performanse pri malom i srednjem opterećenju. Za 50 i 400 istovremenih korisnika prosečno vreme odziva ostaje ispod 8 ms. Ovakvo ponašanje je očekivano s obzirom na to da se podaci keširaju u Redis-u i da se pri većini zahteva ne ostvaruje direktan pristup bazi podataka.

Povećanjem broja korisnika na 750 dolazi do umerenog rasta vremena odziva, pri čemu prosečno vreme odziva ostaje relativno nisko, dok vrednosti 95. percentila rastu brže. Ovo ukazuje na pojavu zagušenja resursa i povećano čekanje pojedinih zahteva u

redovima za obradu, iako sistem i dalje uspešno obrađuje zahteve bez značajnog pada propusnosti.

Pri ekstremnom opterećenju od 2000 istovremenih korisnika uočava se naglo pogoršanje performansi, sa prosečnim vremenom odziva od 882 ms i 95. percentilom koji dostiže 7,3 sekundi. U ovom režimu rada sistem dostiže granice svojih mogućnosti, što rezultuje zadržavanjem zahteva i zasićenjem serverskih resursa..

# Dobavljanje gradova po državi

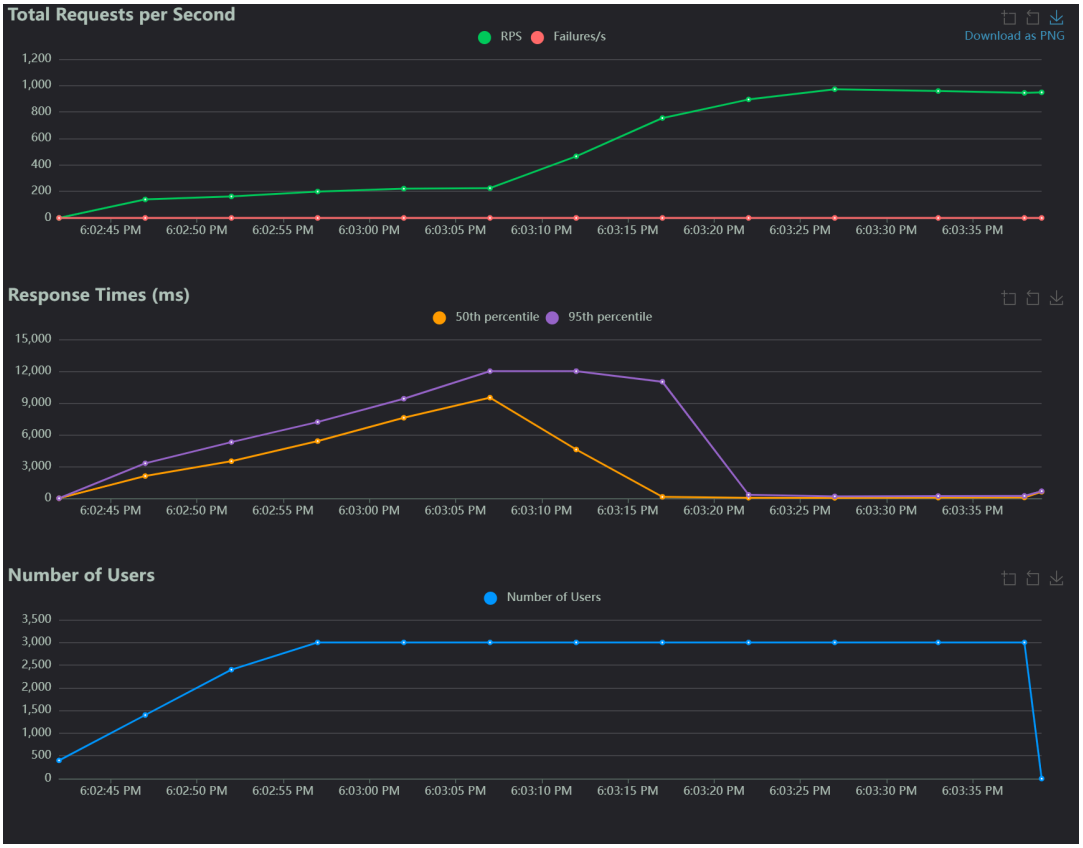
## Endpoint: /api/v1/countries/{id}/cities
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje gradova po državi se automatski poziva pri selekciji države u formi za registraciju firme, kreiranju, ažuriranju i pretrazi fabrika, kreiranju, ažuriranju i pretrazi magacina.

S obzirom na to da se podaci o državama veoma retko menjaju, prilikom prvog poziva svi podaci se upisuju u Redis keš sa vremenom važenja (engl. *TTL*) od 12 sati. Na taj način se smanjuje opterećenje baze podataka i obezbeđuje brz odgovor sistema za veliki broj istovremenih korisnika.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 34,23 | 47,27 |
| 50 | 5 | 15,21 | 20,48 |
| 750 | 20 | 15,05 | 78,55 |
| 1250 | 50 | 44,07 | 284,04 |
| 3000 | 200 | 1224,82 | 9146,81 |

Tabela 12 - rezultati dobavljanja svih gradova




Grafik 8 - grafik dobavljanja gradova po državi (3000 korisnika)

Rezultati testiranja endpoint-a za dobavljanje gradova po državi pokazuju zanimljivo ponašanje u uslovima rastućeg opterećenja. Pri malom broju korisnika prosečno vreme odziva je relativno veće, što se može pripisati inicijalnom punjenju keša i manjem broju ponovljenih zahteva za istim podacima.

Povećanjem broja istovremenih korisnika dolazi do smanjenja prosečnog vremena odziva, što ukazuje na efekat zagrevanja keša i povećan broj pogodaka u kešu.

Pri većem opterećenju, od 1250 korisnika, uočava se postepeni rast vrednosti 95. percentila vremena odziva, što ukazuje na pojavu zadržavanja pojedinih zahteva usled ograničenih serverskih resursa. Pri ekstremnom opterećenju od 3000 korisnika dolazi do značajnog pogoršanja performansi, sa izraženim rastom prosečnog vremena odziva i 95. percentila, što ukazuje na zasićenje sistema i dostizanje granica obrade zahteva.

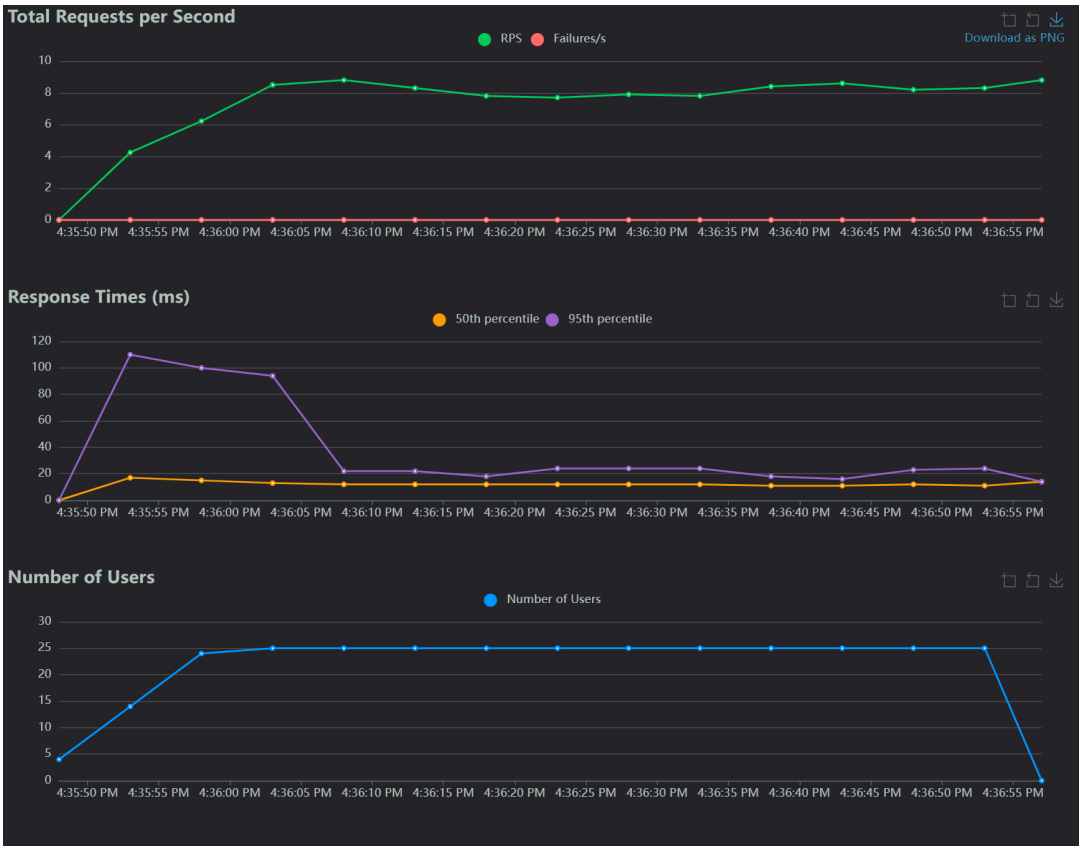
# Verifikacija registrovane firme

Endpoint: /companies/{id}
Metoda: PATCH
Testirano na mašini 1

Endpoint za verifikaciju firme ima ograničen broj entiteta u stanju *PENDING* i svaka firma može biti verifikovana samo jednom. Zbog toga ovaj endpoint nije pogodan za klasično skaliranje broja korisnika. Test je sproveden sa ograničenim brojem konkurentnih korisnika kako bi se simulirala realna administratorska upotreba i proverilo ponašanje sistema prilikom ažuriranja stanja firme. Slanje notifikacionih email poruka realizovano je asinhrono, čime se izbegava blokiranje HTTP zahteva i značajno smanjuje vreme odziva endpointa. Statusni kodovi 400 i 409 se smatraju očekivanim u uslovima konkurentnog pristupa i ne predstavljaju grešku sistema.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 5 | 1 | 22,5 | 24,34 |
| 10 | 1 | 16,77 | 19,32 |
| 25 | 2 | 12,59 | 15,69 |

Tabela 13 - verifikacija registrovane firme




Grafik 9 - grafik verifikacija registrovanih firmi (25 korisnika)

Na osnovu prikazanih rezultata može se uočiti da endpoint za verifikaciju registrovane firme pokazuje dobre performanse pri svakom nivou opterećenja, pri čemu se prosečno vreme odziva i P95 smanjuju sa rastom broja konkurentnih korisnika. Ovakvo ponašanje je očekivano s obzirom na ograničen broj firmi u stanju *PENDING*. Kako se broj validnih verifikacija smanjuje, veći deo zahteva biva odbijen u fazi validacije, što rezultuje kraćim vremenom izvršavanja i nižim vrednostima vremena odziva.

# Dobavljanje neobrađenih firmi

## Endpoint: /companies/pending
## Metoda: GET
## Testirano na mašini 1

Endpoint za dobavljanje neobrađenih firmi koristi se prilikom pregleda firmi od strane menadžera ili administratora. Zbog potencijalno velikog broja zapisa u stanju *PENDING*, rezultati se vraćaju paginirano, čime se smanjuje opterećenje sistema i obezbeđuje stabilno vreme odziva.

Tokom testiranja korišćena je veličina stranice od 10 zapisa, u skladu sa trenutnom frontend implementacijom.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 276,28 | 447,27 |
| 25 | 2 | 436,11 | 838,07 |
| 50 | 5 | 1290,87 | 2136,36 |
| 100 | 5 | 5071,13 | 7618,18 |

Tabela 14 - dobavljanje neobrađenih firmi

Testiranje endpointa pokazalo je značajan rast vremena odziva sa povećanjem broja konkurentnih korisnika. Već pri 100 istovremenih korisnika, P95 vreme odziva prelazi 7 sekundi. Analiza je pokazala da je glavni uzrok opterećenja učitavanje kompleksnog entiteta Company sa više povezanih entiteta (City, County,User, StoredFile) i mapiranje u DTO objekte, što je dovodilo do N+1 problema i prekomernog opterećenja baze podataka.

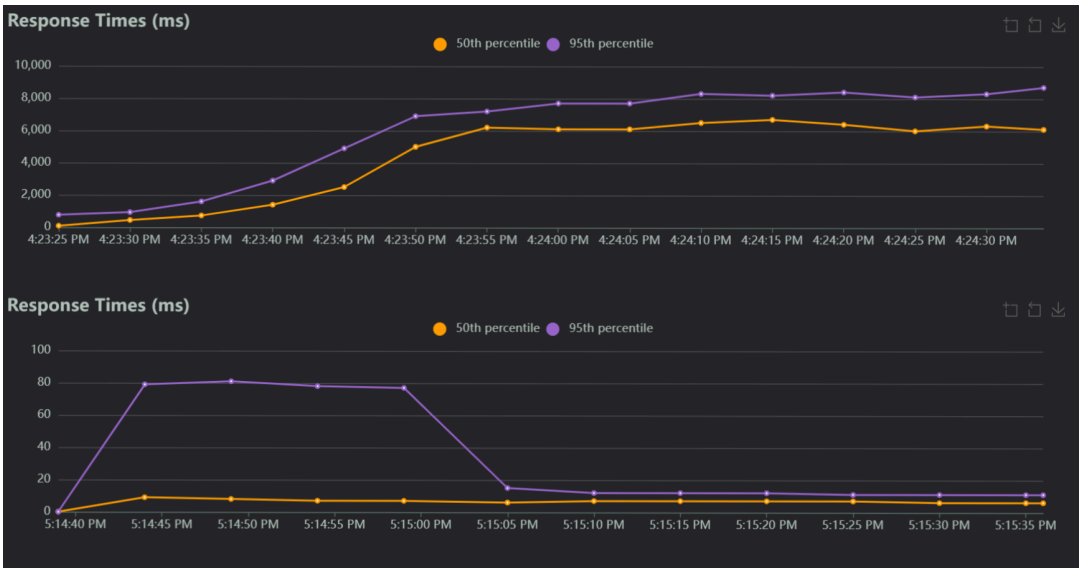
Za rešavanje ovog problema primenjene su sledeće optimizacije:

1. Promena svih EAGER relacija u LAZY kako bi se izbeglo nepotrebno učitavanje povezanih entiteta.

2. Korišćenje DTO projekcija umesto punih entiteta, čime se smanjuje opterećenje Hibernate persistence konteksta.
3. Batch učitavanje fajlova povezanih sa kompanijama, čime se izbegava N+1 problem prilikom prikaza liste fajlova.
4. Dodavanje indeksa na polje status u tabeli Company radi bržeg filtriranja.

| Broj korisnika | Brzina generisanja korisnika | Avg RT (ms) | P95 RT (ms) |
|---|---|---|---|
| 10 | 1 | 12,9 | 20,45 |
| 50 | 5 | 7,95 | 12,73 |
| 100 | 5 | 7,12 | 11,64 |
| 750 | 20 | 14 | 81,1 |
| 3000 | 200 | 1508,16 | 8836,36 |

Tabela 15 - dobavljanje neobrađenih firmi nakon optimizacija




Grafik 10 - poređenje performansi neoptimizovanog i optimizovanog endpointa (100 korisnika)

Na osnovu sprovedenog testiranja i uporedne analize rezultata pre i posle optimizacije može se zaključiti da je glavni uzrok degradacije performansi endpoint-a bio neoptimizovan pristup podacima

Rezultati prikazani u tabelama i grafičkom poređenju potvrđuju da je optimizovani pristup doveo do značajnog poboljšanja performansi, stabilnijeg vremena odziva i linearnog skaliranja sistema do tačke zasićenja.

# Zaključak

Nakon sprovedenih testova opterećenja, može se zaključiti da se sistem dobro skalira za endpointe koji se često čitaju. Najveći uticaj na poboljšanje performansi imalo je uvođenje odgovarajućih indeksa, dok se keširanje pokazalo kao naročito efikasno u scenarijima gde je poznata učestalost pristupa određenim stranicama ili resursima.

Asinhrono slanje elektronske pošte značajno unapređuje korisničko iskustvo tokom procesa registracije i verifikacije kompanija, jer omogućava korisniku da nastavi sa radom odmah nakon uspešnog upisa podataka u bazu. Posebna pažnja posvećena je tome da se slanje mejla izvršava isključivo nakon uspešno završene transakcije, čime se obezbeđuje konzistentnost sistema.

Tokom testiranja identifikovani su i otklonjeni problemi poput N+1 upita, kao i druga uska grla u performansama. Ova uska grla su efikasno ublažena ili u potpunosti uklonjena primenom keširanja i optimizacijom upita kroz indekse.

Važno je napomenuti da su testovi izvršeni u lokalnom okruženju, pri čemu su svi Doker kontejneri bili pokrenuti na istoj fizičkoj mašini. Ovo predstavlja značajno ograničenje testova, budući da su hardverski resursi često bili glavni faktor ograničenja sistema. Zbog toga u pojedinim scenarijima nije bilo moguće dovesti sistem do potpunog zasićenja ili pada pri velikom broju konkurentnih korisnika.

# Appendix

## Bekend konfiguracija tokom testiranja

```
spring.application.name=eurosupply
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

jwt_secret=${SECRET}
jwt_expires=259200000
auth_header=Authorization

spring.jpa.defer-datasource-initialization=true
spring.jpa.hibernate.ddl-auto = create-drop
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

spring.sql.init.data-locations=classpath:data.sql
spring.sql.init.mode=always

file-storage-path=src/main/resources/files/

spring.web.resources.add-mappings=true
springdoc.swagger-ui.path=/swagger-ui.html

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.mode=HTML
spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.cache=false

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

```
# Temporary workaround for Spring AMQP 4.0.0 bug where
SimpleAmqpHeaderMapper.toHeaders()
# throws NullPointerException when message priority is null (line 155).
logging.level.org.springframework.amqp.support.SimpleAmqpHeaderMapper=ERROR

influxdb.host=localhost
influxdb.port=8086
influxdb.organization=eurosupply
influxdb.token=${INFLUX_DB_TOKEN}

spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.data.redis.password=
spring.data.redis.database=0

spring.data.redis.timeout=60000
spring.cache.type=redis

logging.level.org.springframework.cache=DEBUG
logging.level.org.springframework.data.redis=DEBUG

app.base-url=http://localhost:5173
app.email.from=eventorium.info@gmail.com

app.frontend-url=http://localhost:8787
```